VISION / NEXT PHASE

ESP32-CAM FIREFIGHTER

From a line follower that puts out candles to a robot that maps a room, patrols it and refills itself.

Afonso Teixeira · LETI, ISEP

1. Where the robot is today

The current robot is a four-motor car built around an ESP32-CAM, a PC and a Raspberry Pi 5. It already does the following:

- **Drives** from an Xbox controller over WiFi, or **follows a line** of tape on its own using the ESP32 camera and OpenCV on the PC.
- **Detects flames** with FireNet, a small neural network trained on images recorded by the robot, running on the Pi.
- **Stops the car** through a GO/STOP wire from the Pi to the ESP32 when a flame is seen.
- **Aims** a pan/tilt nozzle at the flame using a calibration table, and **sprays** short water pulses with a limited number of retries.
- **Resumes** the line once the scene is clear.

Building it showed clearly what the next version has to change. Each section below starts from a problem seen on the real robot.

2. What has to change, and why

Problem seen	Consequence	Direction for the next phase
The robot can only follow a line of tape	It cannot patrol a real room, go to a flame or find its way back	SLAM: build a map and navigate in it (section 3)
Hundreds of loose wires	Hard to debug, connectors come loose, difficult to reproduce	A custom PCB (section 7)
The chassis bends in the middle	Heavy parts (battery, water, Pi) sit between the axles	New chassis with the weight over the wheels (section 8)
To turn left it stops the left motors and drives the right ones	Wide turns, no turning on the spot, struggles in sharp corners	Independent direction per side, or real steering (section 5)
The car often got stuck with the motors on	As the battery drains, the same PWM no longer moves the car	Wheel encoders and closed-loop speed control (section 6)
FireNet sees reflections of the flame on wet floors	After spraying, the robot stops again for a reflection	A thermal camera to confirm real heat (section 4)
The robot does not know how much water it has	It can spray an empty tank	A water level sensor and a refill station (sections 11 and 12)
The car vibrates a lot while driving	The camera shakes: blurred images, missed or false detections, and a moving aim	Damping, and a camera stabilised with a gyroscope and servos (section 9)
It stops for every flame before spraying	Slow patrols; every flame costs a full stop, aim and restart	Spray while driving (section 10)
It needs light to work	Line following and FireNet fail in the dark	LiDAR for SLAM, infrared or thermal vision for fire (sections 3 and 4)

Table 1

3. Navigation: from following a line to SLAM

SLAM (Simultaneous Localisation and Mapping) lets the robot build a map of the room while working out where it is on that map. With a map, the robot can patrol, go to a point, avoid obstacles and come back to a station, which a line can never give it.

3.1 LIDAR OR CAMERA

Option	How it works	Works at night?	Verdict
2D LiDAR (e.g. LDROBOT LD19 / LD06, Slamtec RPLIDAR C1)	A spinning laser measures the distance to the walls all around the robot, about 10 times per second	Yes: it brings its own light	Recommended: the most reliable and best supported option
Camera, visual SLAM (e.g. ORB-SLAM3, RTAB-Map)	Tracks visual features between frames	Only with infrared light and a NoIR camera	Possible, but needs texture and light, and uses much more CPU
Depth camera (e.g. Intel RealSense, OAK-D)	Measures depth per pixel	Partly (active infrared)	Powerful, but expensive and heavy for this size of robot

Table 2

Recommendation: a 2D LiDAR for the map and navigation, combined with wheel encoders (section 6) and an IMU (e.g. BNO085) so the robot knows how it moved between scans. LiDAR solves the night requirement on its own, because it does not depend on room light.

3.2 SOFTWARE

- **ROS 2** on the Raspberry Pi 5 as the framework that connects sensors, map and motors.
- **slam_toolbox** to build the map from the LiDAR, **Nav2** to plan paths and drive along them.
- The ESP32 (or a new motor microcontroller) receives speed commands and returns wheel speeds, instead of receiving MOV commands from a PC.
- The PC becomes optional: a laptop is only needed to watch the map and give missions.

4. Seeing fire: reflections and the dark

4.1 WHAT HAPPENED WITH REFLECTIONS

After the first water pulse the floor around the candle gets wet, and the wet floor reflects the flame. FireNet only looks at the colour image, and a reflection of a flame looks exactly like a flame, so the robot detected a second "flame" next to the one it had just sprayed and stopped again.

The current fix is a filter, not a solution: after spraying, new detections within a radius of the sprayed position are ignored for a few seconds after the car is allowed to move again, unless the same detection already existed before spraying. It works most of the time, but a real second flame close to the first can be ignored, and a reflection farther away is not filtered.

4.2 A THERMAL CAMERA

A reflection is cold; a flame is hot. A thermal camera measures temperature in every pixel, so it can confirm that a detection is real heat. It also works in complete darkness.

Sensor	Resolution	Notes
Melexis MLX90640	32 × 24 pixels	I2C, connects directly to the Pi. Low resolution, but enough to confirm "there is something hot in that direction".
FLIR Lepton 3.5 (on a breakout board)	160 × 120 pixels	Real thermal image with temperature per pixel. Much more expensive.
Small USB thermal cameras (InfiRay / Topdon modules, 256 × 192)	256 × 192 pixels	Good resolution; check that Linux drivers exist before buying.

Table 3

Proposed use: keep FireNet on the colour camera to find and aim at the flame, and require the thermal camera to see a hot spot in the same direction before the car stops or sprays. That removes the reflection problem at its source and makes the time-based filter unnecessary.

> NOTE Thermal cameras do not see through glass, and a hot object that is not a flame (a radiator, a lamp) is also hot. Colour and heat together are much stronger than either alone.

4.3 FIRE DETECTION AT NIGHT

At night a candle is the brightest thing in the room, so the colour camera still sees it, but everything else disappears. The thermal camera covers detection; for the navigation the LiDAR covers the map. If a camera is still wanted for the map, a NoIR camera (no infrared filter) with an infrared LED illuminator sees in the dark without lighting the room for people.

5. Turning

Today both L293D drivers share the same direction pins, so the two sides of the car always turn in the same direction. To turn left, the left motors are slowed or stopped and the right ones keep going. The car turns in a wide arc and cannot turn on the spot.

Option	What changes	For	Against
A. Independent direction per side (skid steer / tank)	One direction signal per side: one more GPIO, or a separate motor controller	Turns on the spot; standard for ROS 2 (diff_drive); no new mechanics	Wheels skid when turning, which makes encoder odometry less precise
B. Car-like steering (Ackermann)	A servo turns the front wheels; the rear wheels drive	Smooth turns, less tyre wear	Cannot turn on the spot, larger turning radius, new mechanics and harder navigation

Table 4

Recommendation: option A. It needs almost no mechanical change and suits SLAM navigation in tight rooms. The ESP32-CAM has no free pins left, which is one more reason to move the motors to a separate controller on the new PCB (section 7).

6. Knowing the wheels really turn

The motors are controlled in open loop: the robot sends a PWM value and assumes the wheels turn. As the battery drains, the same value produces less force, and the car can sit still with the motors buzzing indefinitely.

Encoders fix this. Each motor gets a sensor that counts wheel rotations. With them the robot can:

- Control **speed instead of PWM**: a PID loop raises the PWM until the wheel reaches the requested speed, so a weaker battery is compensated automatically.
- **Detect a stall:** the command is high but the wheel does not turn. The robot then tries more power, then backs up, and if it is still stuck it stops and reports it instead of burning the motors.
- Measure **how far it moved**, which SLAM needs (odometry).

Option	Notes
Gear motors with built-in Hall encoders (e.g. JGA25-370 or N20 with encoder)	Recommended: encoder already aligned with the motor, two channels give direction too
Optical slot sensor and slotted disc on the existing wheels	Cheapest way to keep the current motors; one channel only, lower resolution
Current sensor (e.g. INA219) per side	Extra: a stalled motor draws much more current, a second way to detect being stuck

Table 5

The L293D also wastes about 2 V of the battery as heat. A modern MOSFET driver such as the TB6612FNG or DRV8833 gives the motors more of the battery voltage and runs cooler.

7. A PCB instead of wires

The current robot is built on protoboards with hundreds of wires. A custom PCB, designed in KiCad and ordered from a PCB manufacturer, would hold:

- Power: battery input with a fuse and reverse-polarity protection, the 5 V supply for the Pi and servos, and a voltage divider so the robot can measure its own battery.
- Motor drivers (TB6612FNG or DRV8833) and a microcontroller for the motors and encoders (an ESP32-S3 or RP2040).
- The pump MOSFET with its flyback diode, and the gate resistors already in place.
- Locking connectors (JST) for motors, servos, pump, sensors and LiDAR, and a header for the Raspberry Pi.

Fewer wires means fewer intermittent faults, and the design can be documented, shared and rebuilt exactly.

8. A chassis that does not bend

The printed chassis bends in the middle because the heaviest parts — battery, water and Raspberry Pi — sit between the axles, where the plate has the least support. For the next version:

- Put the battery and the water tank over or between the wheels, as low as possible, so the weight goes straight into the axles.
- Add ribs or a box section along the length of the chassis instead of a flat plate.
- Print in PETG or nylon instead of PLA, or use an aluminium or carbon plate as the base with printed parts on top.
- Check the design with a simple load simulation in Fusion 360 before printing.

9. Vibration and a stabilised camera

The car shakes a lot while it drives, and the camera shakes with it. The images blur, small flames are missed or appear in the wrong place, and the aim moves even when the servos do not. This has two parts: make the car vibrate less, and keep the camera still when it still does.

9.1 VIBRATING LESS

- Mount the camera and the nozzle on **rubber dampers** (silicone grommets or anti-vibration balls, like those used on drone cameras), not screwed straight to the chassis.
- A **stiffer chassis** (section 8) vibrates less than a plate that bends in the middle.

- Check that the **wheels run true** and are tight on their shafts; a wobbling wheel shakes the whole car on every turn.
- Softer tyres absorb small bumps from the floor.
- **Smoother motor control:** with encoders and a speed loop (section 6) the motors no longer jump between stopped and full power, which is a large part of the shaking. A higher PWM frequency (around 20 kHz with a MOSFET driver) also makes the motors run smoother and quieter.

9.2 A CAMERA THAT ALWAYS LOOKS THE SAME WAY

A **gimbal** keeps the camera pointing in the same direction while the car moves under it. An IMU (gyroscope and accelerometer) on the camera mount measures how the mount tilts and turns, and the motors immediately turn the camera the opposite way. It is the same principle as the stabilisers used for phones and drones.

Option	How it works	Notes
IMU + the current servos	The Pi reads the IMU and corrects pan and tilt in software	Cheapest, and reuses the pan/tilt already on the robot. SG90 servos are slow and have play, so they can only cancel slow movements, such as tilting on a slope, not fast vibration.
IMU + metal-gear digital servos (e.g. MG90S)	Same, with faster and more precise servos	Better, still simple. The fastest shaking still gets through.
Brushless gimbal (2 or 3 axes, with its own controller board)	A dedicated controller reads its IMU hundreds of times per second and drives brushless motors	Recommended for real stabilisation. The Pi only says where to point; the gimbal holds that direction by itself.

Table 6

Recommendation: dampers first, because they are cheap and remove the fast vibration that no servo can follow. Then a brushless gimbal carrying both the camera and the nozzle, so that aiming and stabilisation become one system: the Pi chooses the direction, the gimbal keeps it.

10. Spraying while driving

Today the robot stops for every flame, settles, aims, sprays and only then drives on. In the next phase the goal is to keep moving during the search: when a flame is confirmed, the robot sprays it while it drives past, and only stops if it keeps missing.

10.1 WHAT MAKES IT POSSIBLE

- A **stabilised turret** (section 9): the nozzle has to stay on target while the chassis moves and shakes under it.
- **Knowing how the robot moves:** encoders and the IMU give its speed and rotation, and SLAM gives its position, so the aim can compensate for the car's own movement instead of chasing the flame.
- **Leading the target:** the water takes a moment to reach the flame (about 0.3 s at 1 m, from the jet model in the report). During that time the car keeps moving, so the nozzle must point where the flame will be when the water arrives, not where it is now.
- **Flame position on the map:** with the LiDAR distance and the camera direction, each flame gets a fixed position in the room. Aiming at a fixed point on the map is much easier than aiming at a point in a moving image.
- **The thermal camera** (section 4) to confirm hits: a flame that is out stops being hot, which tells the robot whether to try again.

10.2 HOW THE ROBOT DECIDES

Situation	Behaviour
Flame confirmed, clear shot	Slow down, track it with the turret and spray without stopping.
Still burning after the pass	Turn back and try again, or stop and use the current stop-aim-spray method.
Several flames in view	Spray the nearest first, then the next, without stopping between them.

Table 7

> **TRADE-OFF** Spraying on the move is faster but less accurate than stopping. Keeping stop-and-aim as a fallback means the robot never gives up on a flame just because a moving shot missed.

11. Water level

Sensor	What it measures	Notes
Non-contact capacitive sensor (e.g. XKC-Y25)	Water present or not at one height	Glued outside the tank, nothing touches the water. Two of them give "low" and "full".
Float switch	Water above or below one level	Simple and cheap, but moving parts inside the tank.
Waterproof ultrasonic sensor (e.g. JSN-SR04T)	Continuous level	Gives a percentage, but needs some height above the water.
Flow sensor (e.g. YF-S401) on the pump outlet	Water used	Counts how much was sprayed; combined with a level sensor it detects a blocked hose.

Table 8

Recommendation: two capacitive sensors (low and full). They are reliable, have no moving parts and are enough for the mission logic: "full", "enough", "go and refill".

12. The refill station

A fixed station where the robot can refill its tank on its own:

- **Position:** the station has a fixed place on the SLAM map. For the last centimetres, an ArUco/AprilTag marker seen by the camera aligns the robot precisely.
- **Docking detection:** a limit switch or a Hall sensor and magnet confirms the robot is in place before any water flows.
- **Filling:** the station has its own tank and a solenoid valve (or a small pump) above a funnel on the robot. A small ESP32 on the station opens the valve when the robot asks for it over WiFi, and closes it when the robot's "full" sensor triggers.
- **Optional later:** charging contacts (pogo pins) on the same dock so the robot can also charge.

13. The mission

With a map, a water level and a station, the robot's behaviour becomes a mission instead of a single reaction:

Situation	What the robot does
Patrolling	Follows a route through the map, looking for flames with the colour and thermal cameras.
Flame confirmed, water available	Sprays while driving past (section 10); stops and aims only if it keeps missing. Then continues the patrol.
Water low	Stops patrolling and plans a path to the refill station.
Flame seen on the way to the station	Marks the flame's position on the map (robot position plus the direction and distance from the camera and LiDAR) and keeps going to refill.
After refilling	Goes to the marked flames first, nearest first, before resuming the patrol.
Stuck or lost	Reports it and waits, instead of driving the motors indefinitely.

Table 9

14. Materials

Item	Purpose	Example
2D LiDAR	Map and navigation, day and night	LDROBOT LD19 or Slamtec RPLIDAR C1
IMU	Rotation and acceleration for odometry	BNO085 or MPU-6050
Thermal camera	Confirm real heat, reject reflections, see at night	MLX90640 (cheap) or FLIR Lepton 3.5
NoIR camera + infrared LEDs (optional)	Camera image in the dark	Raspberry Pi Camera Module 3 NoIR
Gear motors with encoders (×4)	Speed control and stall detection	JGA25-370 with Hall encoder
Motor drivers	Replace the L293D	TB6612FNG or DRV8833
Motor microcontroller	Motors, encoders, sensors; frees the camera board	ESP32-S3 or RP2040
Current sensors (optional)	Second way to detect a stall	INA219
Water level sensors (×2)	Know when to refill	XKC-Y25 capacitive
Flow sensor (optional)	Water sprayed, blocked hose	YF-S401
Custom PCB	Replace the wiring	KiCad design, JST connectors
Chassis material	Stiffer base	PETG / nylon, or aluminium plate
Station: tank, solenoid valve, ESP32, funnel	Refilling	12 V solenoid valve, small ESP32 board
Station: docking sensor and marker	Know the robot is in place	Limit switch or Hall sensor and magnet; printed AprilTag
Rubber dampers	Isolate camera and nozzle from vibration	Silicone anti-vibration balls or grommets
Gimbal	Keep camera and nozzle pointing at the target	2/3-axis brushless gimbal with controller, or IMU + MG90S servos
Battery	More capacity for LiDAR and Pi	Larger 2S pack with a BMS

Table 10

Choose the exact parts after checking current availability, voltages and Linux/ROS 2 support.

15. Estimated cost

Approximate prices in euros for the new parts only, from typical online shops (cheaper from AliExpress, dearer from European distributors). The Raspberry Pi 5, servos, pump and cameras already on the robot are not counted.

Item	Approx. price
2D LiDAR (LD19 / RPLIDAR C1)	70 – 110 €
IMU (MPU-6050 or BNO085)	5 – 30 €
Thermal camera: MLX90640	50 – 80 €
Thermal camera: FLIR Lepton 3.5 with breakout (instead of MLX90640)	250 – 350 €
4 gear motors with encoders (JGA25-370)	40 – 60 €
2 motor drivers (TB6612FNG / DRV8833)	5 – 10 €
Motor microcontroller (ESP32-S3 / RP2040)	5 – 15 €
2 water level sensors (XKC-Y25)	10 – 20 €
Custom PCB (5 boards) + components and connectors	35 – 70 €
Chassis material (PETG, aluminium plate, screws)	25 – 50 €
Refill station (tank, 12 V solenoid valve, ESP32, power supply, dock sensor)	40 – 60 €
Larger 2S battery with BMS	25 – 50 €
Rubber dampers for camera and nozzle	5 – 10 €
Brushless gimbal with controller (or IMU + 2 MG90S servos: 10 – 20 €)	30 – 70 €
Wires, connectors and small parts	20 – 30 €
Optional: NoIR camera + IR LEDs, current sensors, flow sensor	35 – 65 €

Table 11

Total: about **370 – 670 €** with the MLX90640 thermal sensor and without the optional items; about **690 – 1 000 €** with the FLIR Lepton and everything optional. Prices change often; check them before ordering.

16. Suggested order

Step	Work	Why at this point
1	Encoders, new motor drivers and closed-loop speed	Everything else depends on the robot moving reliably
2	Independent direction per side	Small change, needed for navigation
3	LiDAR, IMU and SLAM with ROS 2	Build and save a map, then drive to points on it
4	Thermal camera and fusion with FireNet	Fixes reflections, adds night detection
5	Dampers and a stabilised camera/nozzle gimbal	Sharper images and a steady aim; needed before spraying on the move
6	Water level sensors	Simple, and the mission needs them
7	Refill station and the mission logic	Needs map, navigation and water level working
8	Spraying while driving	Needs the gimbal, odometry and flame positions on the map
9	PCB and new chassis	Once the parts are chosen and tested on protoboard

Table 12

17. Servos with readable position feedback

Problem observed. A bump or chassis vibration can start the current SG90 pan/tilt servos shaking. Sometimes the oscillation continues after the initial disturbance, moving the camera and nozzle and preventing a stable aim.

Planned change. Replace the current servos with units that expose their measured shaft position to the controller, through a feedback output or a readable digital interface. The controller should know where each servo actually is, rather than assume that the requested PWM pulse has already placed it there.

How this helps. Compare the requested and measured positions, detect repeated oscillation or a stalled movement, and wait for the mount to settle before spraying. Use limited correction speed and a deadband so small errors do not trigger constant adjustments.

What still needs checking. The SG90 already has internal position control, but its normal three-wire connection does not report position to the Raspberry Pi. External feedback alone will not fix loose supports, gear play, an unsuitable power supply or unstable control tuning. Address those causes too; use an IMU to measure camera orientation, because shaft angle alone cannot measure chassis movement.

Integration. Include this replacement in the stabilisation work in section 9 and step 5 above. Select the interface, torque, voltage and mechanical fit before updating the materials list and budget; the existing cost estimate does not include these replacement servos.